Contents

A Supplementary Sweep Results **2**
A.1 Sweep Configuration Space 2
A.2 XGBoost Configuration 2
A.3 Compute Environment for Reproducibility 3

B User Guide **4**
B.1 Prerequisites 4
B.2 Installation 5
B.3 Downloading the Source Data 5
B.4 Configuration 6
B.5 Pipeline Walkthrough 6
B.6 Hyperparameter Sweep 7
B.7 Running Tests 9

C Source Code **10**
C.1 How to Navigate the Repository 10
C.2 High-Value Modules by Question 12
C.3 Reproducibility Anchors 13

Appendix A

Supplementary Sweep Results

This appendix provides additional hyperparameter sweep visualisations, sweep budgets, and the documented compute environment for runs whose quantitative findings appear in the main report's **Chapter 7 (Evaluation/Results)**. It does not reproduce those headline results, which are argued there; the Implementation chapter (Chapter 6) describes code structure only. Operational commands appear in the User Guide and Source Code appendices.

A.1 Sweep Configuration Space

The sweep budgets are dataset-specific. This was an explicit practical decision: the MIMIC run was computationally expensive in wall-clock time, so the Challenge 2012 profile used a smaller full-grid budget to keep the overall experimental campaign feasible.

Setting	**MIMIC full profile**	**Challenge 2012 full profile**
n_t values	$\{1, 2, 3, 5, 8, 13, 21, 34\}$ (8 values)	$\{1, 2, 3, 5, 8, 13, 21\}$ (7 values)
n_{noise} values	$\{1, 2, 3, 5, 8, 13, 21, 34\}$ (8 values)	$\{1, 2, 3, 5, 8, 13, 21\}$ (7 values)
Grid size	$8 \times 8 = 64$ cells	$7 \times 7 = 49$ cells
Max training rows	100,000	100,000
Synthetic samples per cell	100,000	50,000
Privacy metric row cap	2,000	2,000

Table A.1: Dataset-specific full-profile sweep budgets from `src/lexisflow/config/datasets.py`.

Unless otherwise stated, sweeps in this appendix use the HS3F backbone and three trajectory sampling seeds per cell (`TSTR_TRAJECTORY_SAMPLING_SEEDS = (42, 11, 50)`).

A.2 XGBoost Configuration

The shared sweep XGBoost parameter block (`SWEEP_XGB_PARAMS`) is:

Parameter	Value
`n_estimators`	100
`max_depth`	6
`learning_rate`	0.1
`subsample`	0.8
`colsample_bytree`	0.8

Table A.2: Shared XGBoost hyperparameters used in sweep training.

These parameters were not tuned as part of the sweep; they represent a single fixed operating point. Parallelism is controlled separately by the sweep CLI `--n-jobs` (default `8`), and model random state is fixed in the sweep factory for reproducibility.

A.3 Compute Environment for Reproducibility

All reported sweeps were executed on a local Apple Silicon workstation. The machine characteristics are listed for transparency because runtime was a material constraint in sweep design:

Component	Specification
CPU	Apple M1 Max
Logical CPU cores	10
Memory	32 GB RAM
Operating system	macOS 26.4 (Darwin 25.4.0, arm64)
Typical sweep parallelism	`--n-jobs 8` (varied by run)

Table A.3: Hardware and software environment used for the reported sweeps.

The 32 GB figure is the machine on which results in Chapter 7 were produced; it does *not* demonstrate that the same sweep completes on a smaller-RAM workstation. The main report's memory-related NFR is therefore stated as a design intent, not a verified minimum-RAM guarantee.

Appendix B

User Guide

This appendix is the full operational reference for the pipeline (install, configuration, single-command sweeps, resumability, and result paths). The main report's Chapter 6 gives only the minimum needed to interpret methods and results; details that would duplicate the argumentative spine are concentrated here to avoid bloat in the main volume. All commands assume a Unix-like terminal (macOS or Linux) and are run from the project root directory.

B.1 Prerequisites

- **Python 3.10 or later.**
- **uv** (recommended) or **pip** for dependency management. Install uv via:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

- **MIMIC-III access.** A credentialed PhysioNet account with a signed data use agreement is required to obtain the source data. See `https://mimic.physionet.org/gettingstarted/access/`.
- **No credentialed access available?** If you cannot access the credentialed MIMIC-III dataset, use the public Challenge 2012 workflow documented in section B.6.1 instead.
- **Preprocessed MIMIC tables.** The pipeline expects hourly ICU tables produced under the preprocessing contract in section 5.1.1, placed under the project data directory.

B.2 Installation

Clone the repository and install dependencies:

```
git clone https://github.com/lianmatsuo/lexisflow
cd lexisflow
uv sync            # installs all dependencies from pyproject.toml
```

Alternatively, with pip:

```
pip install -e .
```

To verify the installation, run the test suite:

```
uv run pytest
```

All unit tests should pass.

B.3 Downloading the Source Data

B.3.1 MIMIC-III (Credentialed)

1. Create or sign in to a PhysioNet account.
2. Complete the required credentialing steps and data use agreement for MIMIC-III.
3. Request access and, once approved, download the required MIMIC-III tables from PhysioNet: `https://physionet.org/content/mimiciii/1.4/`.
4. Run the project preprocessing pipeline to produce the expected derived CSV inputs used by this repository (hour-0 and autoregressive tables).

B.3.2 PhysioNet Challenge 2012 (Public)

If you do not have credentialed MIMIC-III access, use the public Challenge 2012 dataset: `https://physionet.org/content/challenge-2012/1.0.0/`.

1. Download the Challenge 2012 training set files from the PhysioNet page above.
2. Place raw files under the repository's Challenge 2012 data area (project `data/challenge2012/` tree).
3. Run:

   ```
   uv run python scripts/run_sweep.py --dataset challenge2012 --reset --n-jobs 8
   ```

This command prepares processed CSVs and executes the full Challenge 2012 sweep.

B.4 Configuration

Runtime configuration is now centralised in Python dataset presets plus CLI overrides. The authoritative defaults are defined in `src/lexisflow/config/datasets.py`.

- **Dataset paths (`DatasetConfig`):** script directory, processed CSV paths, preprocessor artifact paths, transformed cache location, results CSV, and sweep artifact directory.
- **Sweep defaults (`SweepDefaults`):** `nt` grid, `n_noise` grid, training row cap, synthetic sample count, and privacy-evaluation row cap.
- **Split policy (`SplitConfig`):** patient-level test fraction, holdout fraction, and split shuffle seed.
- **Profiles (`sweep_profiles`):** each dataset exposes `full` and `smoke` presets. Scripts resolve defaults from the selected profile, then apply explicit CLI overrides.

Practical rule: treat `datasets.py` as the single source of truth for default paths, sweep budgets, and split ratios.

B.5 Pipeline Walkthrough

The pipeline is now executed through a single command that orchestrates data preparation, preprocessor fitting, and the sweep run.

B.5.1 Single Command Pipeline

```
uv run python scripts/run_sweep.py --dataset mimic
uv run python scripts/run_sweep.py --dataset challenge2012
uv run python scripts/run_sweep.py --dataset mimic --reset
uv run python scripts/run_sweep.py --dataset mimic --profile smoke
uv run python scripts/run_sweep.py --dataset mimic --n-jobs 8
```

Outputs (MIMIC):

- `data/processed/hour0_data.csv`
- `data/processed/autoregressive_data.csv`
- `artifacts/hour0_preprocessor.pkl`

- `artifacts/preprocessor_full.pkl`
- `results/sweep_results.csv`

Outputs (Challenge 2012):

- `data/challenge2012/processed/hour0_data.csv`
- `data/challenge2012/processed/autoregressive_data.csv`
- `artifacts/challenge2012/hour0_preprocessor.pkl`
- `artifacts/challenge2012/autoregressive_preprocessor.pkl`
- `results/challenge2012_sweep_results.csv`

B.5.2 Step 4: Evaluate Quality

Quality metrics are computed automatically for every sweep cell (KS statistics, correlation Frobenius norm, clinical range violations, privacy diagnostics). Use `scripts/run_sweep.py --dataset mimic` (or `--dataset challenge2012`) for end-to-end execution.

B.6 Hyperparameter Sweep

For systematic evaluation across configurations, use the unified sweep entrypoint:

```
uv run python scripts/run_sweep.py --dataset mimic
uv run python scripts/run_sweep.py --dataset challenge2012
uv run python scripts/run_sweep.py --dataset mimic --reset
```

Key options:

- `--dataset`: choose `mimic` or `challenge2012`.
- `--profile`: choose `full` or `smoke`; forwarded to the dataset-specific sweep driver.
- `--n-jobs`: override CPU worker count used during the sweep stage.
- `--reset`: clear generated data, preprocessors, cache and sweep outputs before rerun.

The sweep trains both hour-0 and autoregressive models for each (n_t, n_{noise}) configuration, generates synthetic samples, runs multi-task TSTR and quality/privacy metrics, and appends to the dataset-specific results CSV (`results/sweep_results.csv` for MIMIC, `results/challenge2012_sweep_results.csv` for Challenge 2012). The sweep is resumable: existing rows are skipped on rerun.

B.6.1 Challenge 2012 Sweep Quick Guide

Use this sequence when you specifically want the public-benchmark run:

1. **Fresh run (recommended first pass):**

   ```
   uv run python scripts/run_sweep.py --dataset challenge2012 --reset --n-jobs 8
   ```

   This rebuilds processed CSVs, preprocessors, transformed cache, model artifacts, and `results/challenge2012_sweep_results.csv`.

2. **Resume without deleting prior work:**

   ```
   uv run python scripts/run_sweep.py --dataset challenge2012 --n-jobs 8
   ```

   Existing `(nt, n_noise)` rows are skipped automatically.

3. **Low-cost smoke validation:**

   ```
   uv run python scripts/run_sweep.py --dataset challenge2012 --profile smoke
   ```

Important CLI distinction. `--reset` is a flag of the *top-level* driver (`scripts/run_sweep.py`), not the dataset-specific `scripts/challenge2012/run_sweep.py`. Use the top-level command when you want artifact cleanup before rerun.

How to read Challenge 2012 outputs. The main file is `results/challenge2012_sweep_results.csv`. In this CSV:

- `error` contains failure messages for interrupted/failed cells; successful rows have an empty error field.
- `los_synth_roc_auc` and `los_real_roc_auc` are valid when the run uses the updated LOS ROC-AUC fallback path for two-class splits.
- `stay_length_ks` should be interpreted carefully on Challenge 2012 because both real and synthetic tables are represented as fixed 48-hour windows in this pipeline.
- `range_violation_pct` can be influenced by source-data outliers (not only generator error), especially in blood-pressure fields.

Dataset sweep drivers (`scripts/mimic/run_sweep.py` and `scripts/challenge2012/run_sweep.py`) also support direct execution with fine-grained overrides:

- `--profile`, `--train-rows`, `--synth-samples`
- `--nt-values`, `--noise-values`, `--n-jobs`
- `--model-type` (`hs3f`, `forest-flow`, `ctgan`)
- `--refresh-transformed-cache`, `--skip-privacy-metrics`, `--privacy-max-rows`

To visualise sweep results:

```
uv run python scripts/common/analyze_sweep.py
uv run python scripts/common/analyze_sweep.py --dataset challenge2012
uv run python scripts/common/analyze_sweep.py --results-path results/challenge2012_sweep_results.c
```

`analyze_sweep.py` options:

- `--dataset`: selects default results path from dataset config.
- `--results-path`: explicit input CSV override.
- `--output-dir`: explicit plot output directory override.

B.7 Running Tests

The project includes 54 unit tests covering quality metrics, privacy metrics, TSTR framework, autoregressive transformation, feature utilities, model training, and sampling:

```
uv run pytest                    # run all tests
uv run pytest -v                 # verbose output
uv run pytest src/lexisflow/models/   # run model tests only
```

Appendix C

Source Code

Originality Statement

I verify that I am the sole author of the programs contained in this appendix, except where explicitly stated to the contrary.

Lian Matsuo

April 22, 2026

The complete source code is submitted alongside this report. The main volume's Implementation chapter (Chapter 6) documents modelling and evaluation logic; this appendix is the canonical place for repository navigation, run paths, and artefact locations. The task-oriented map below highlights where to start for running sweeps, understanding model behaviour, and validating outputs.

C.1 How to Navigate the Repository

Recommended reading path.

1. Start with the top-level orchestration command in `scripts/run_sweep.py`.
2. Follow dataset defaults and sweep budgets in `src/lexisflow/config/datasets.py`.
3. Read model backbones in `src/lexisflow/models/`.
4. Inspect metrics in `src/lexisflow/evaluation/`.
5. Validate assumptions with tests under `src/lexisflow/*/tests/`.

Core run path (end-to-end).

Stage	Primary file(s)	Purpose
Sweep entrypoint	`scripts/run_sweep.py`	Single command that orchestrates data prep, preprocessing, training, generation, and evaluation.
Dataset-specific drivers	`scripts/mimic/run_sweep.py`, `scripts/challenge2012/run_sweep.py`	Apply dataset profile defaults and execute the full (n_t, n_{noise}) grid.
Model training	`scripts/train_hour0.py`, `scripts/train_autoregressive.py`	Train IID hour-0 model and autoregressive transition model.
Evaluation and plots	`src/lexisflow/evaluation/*.py`, `scripts/common/analyze_sweep.py`	Compute quality/privacy/TSTR metrics and generate sweep visualisations.

C.2 High-Value Modules by Question

If you need to understand...	Go to...	What to look for
How feature types are inferred	`src/lexisflow/data/feature_utils.py`	Rules that classify columns as continuous, binary, or categorical before model fitting.
How autoregressive rows are built	`src/lexisflow/data/autoregressive.py`	Lag construction and static/dynamic feature separation for trajectory modelling.
How ForestFlow is implemented	`src/lexisflow/models/forest_flow.py`	Conditional flow matching over discrete time levels with XGBoost regressors.
How HS3F differs from ForestFlow	`src/lexisflow/models/hs3f.py`	Heterogeneous sequential generation with separate handling of continuous and categorical targets.
How trajectory sampling works	`src/lexisflow/models/sampling.py`	Iterative autoregressive rollout of synthetic patient timelines.
How quality metrics are computed	`src/lexisflow/evaluation/quality_metrics.py`	KS statistics, correlation preservation, and clinical range-violation calculations.
How privacy risk is measured	`src/lexisflow/evaluation/privacy_metrics.py`	DCR-based summaries and DOMIAS-style membership inference implementation.
How downstream utility is measured	`src/lexisflow/evaluation/tstr_framework.py`	Multi-task patient-level TSTR protocol for mortality and LOS prediction.

C.3 Reproducibility Anchors

Artifact type	Location	Why it matters
Sweep defaults	`src/lexisflow/config/datasets.py`	Canonical grid values, training-row budgets, and profile definitions used by both datasets.
Primary results CSVs	`results/sweep_results.csv`, `results/challenge2012_sweep_results.csv`	Per-cell metrics and error fields used to generate plots and report tables.
Preprocessor artifacts	`artifacts/*preprocessor*.pkl`	Capture fitted transforms required for reproducible inverse-transform and generation.
Model artifacts	`artifacts/*/sweep/`	Persist trained models per grid cell for traceability and reruns.
Unit tests	`src/lexisflow/*/tests/`	Regression checks for data transforms, models, and evaluation metrics.
Dependency lock/config	`pyproject.toml`, `uv.lock`	Pin package versions and Python constraints for environment reproducibility.

Note: Complete source code listings are submitted electronically as a companion archive. The program files used in the project examination are identical to the files uploaded to KEATS.